Aziz MANSOUR · 5DS3

Compte rendu: Structuration de l'API et architecture SoC

1. Objectif du TP

Ce TP a pour but de refactoriser un serveur Express monolithique en modules séparés selon le principe de séparation des préoccupations (SoC) : routes, contrôleurs, utilitaires de validation et point d'entrée server.js. Il a été appliqué au domaine users, monté sur /api/users, à côté du domaine articles déjà existant.

Architecture obtenue :

```
my-blog-api/
├── server.js (point d'entrée, middlewares, montage des routeurs)
├── controllers/
│   ├── articleController.js
│   └── userController.js (logique métier et données en mémoire)
├── routers/
│   ├── articleRoutes.js
│   └── userRoutes.js (association verbe HTTP / URL → contrôleur)
└── utils/
    └── validators.js (fonctions de validation réutilisables)
```

2. Réponses aux questions :

Question 1 : Limites d'un serveur monolithique

Pour une application de 10 ressources avec 5 opérations par ressource, le serveur doit gérer environ 50 opérations, en plus de la configuration, des middlewares et des données. Le fichier server.js deviendrait volumineux et difficile à maintenir. Si plusieurs développeurs le modifient en même temps, les risques de conflits Git augmentent.

Question 2 : Réponse après la refactorisation

La réponse de GET http://localhost:3000/api/articles ne doit pas changer après la refactorisation. Les routes et la logique sont déplacées dans des fichiers séparés, mais l'URL et le comportement attendu pour le client restent identiques.

3. Tests Postman : Domaine articles

3.1 PUT réussi : modification de l'article 2

- Requête : PUT http://localhost:3000/api/articles/2
- Corps JSON : { "title": "Express et Architecture SoC Maîtrisés" }
- Résultat attendu : 200 OK


```
{
  "message": "Article mis à jour",
  "article": {
    "id": 2,
    "title": "Express et Architecture SoC Maîtrisés",
    "author": "Aya"
  }
}
```


3.2 PUT en échec : identifiant inexistant

- Requête : PUT http://localhost:3000/api/articles/99
- Résultat attendu : 404 Not Found.

```
{ "error": "Article 99 introuvable" }
```

3.3 DELETE réussi : suppression de l'article 1

- Requête : DELETE http://localhost:3000/api/articles/1
- Résultat attendu : 200 OK

```
{ "message": "Article 1 supprimé avec succès" }
```

4. Tests Postman : Domaine users

4.1 GET : liste complète des utilisateurs

- Requête : GET http://localhost:3000/api/users
- Résultat obtenu : 200 OK

```
{
  "total": 2,
  "users": [
    { "id": 1, "name": "Aziz", "email": "aziz@example.com", "role": "admin" },
    { "id": 2, "name": "Aya", "email": "aya@example.com", "role": "user" }
  ]
}
```

4.2 GET : filtre par rôle

- Requête : GET http://localhost:3000/api/users?role=admin
- Résultat obtenu : 200 OK

```
{
  "total": 1,
  "users": [
    { "id": 1, "name": "Aziz", "email": "aziz@example.com", "role": "admin" }
  ]
}
```

4.3 GET : utilisateur par identifiant

- Requête : GET http://localhost:3000/api/users/2
- Résultat obtenu : 200 OK

```
{
  "id": 2,
  "name": "Aya",
  "email": "aya@example.com",
  "role": "user"
}
```

4.4 GET : identifiant inexistant

- Requête : GET http://localhost:3000/api/users/99
- Résultat obtenu : 404 Not Found

```
{ "error": "Utilisateur 99 introuvable" }
```

4.5 POST : création valide

- Requête : POST http://localhost:3000/api/users
- Corps JSON : { "name": "Sami", "email": "sami@example.com", "role": "user" }

- Résultat obtenu : 201 Created

```
{
  "message": "Utilisateur créé",
  "user": { "id": 3, "name": "Sami", "email": "sami@example.com", "role": "user" }
}
```

4.6 POST : email invalide

- Requête : POST http://localhost:3000/api/users
- Corps JSON : { "name": "Sami", "email": "samiexample.com", "role": "user" }
- Résultat obtenu : 400 Bad Request

```
{ "error": "L'email est obligatoire et doit être valide" }
```

4.7 DELETE : suppression d'un utilisateur

- Requête : DELETE http://localhost:3000/api/users/2
- Résultat obtenu : 200 OK

```
{ "message": "Utilisateur 2 supprimé avec succès" }
```

5. Codes sources

5.1 controllers/userController.js

```
// Données isolées dans le contrôleur (en mémoire pour cette séance)
let users = [
  { id: 1, name: 'Aziz', email: 'aziz@example.com', role: 'admin' },
  { id: 2, name: 'Aya', email: 'aya@example.com', role: 'user' }
];
let prochainId = 3;
const { estNonVide, estEmailValide } = require('../utils/validators');
// 1. Récupérer tous les utilisateurs (avec filtre optionnel ?role=...)
const getAllUsers = (req, res) => {
  const { role } = req.query;
  let resultat = users;
  if (role) {
    resultat = users.filter(u => u.role === role);
  }
  res.status(200).json({ total: resultat.length, users: resultat });
};
// 2. Récupérer un utilisateur par son ID unique
const getUserById = (req, res) => {
  const id = Number(req.params.id);
  const user = users.find(u => u.id === id);
  if (!user) {
    return res.status(404).json({ error: `Utilisateur ${id} introuvable` });
  }
  res.status(200).json(user);
};
// 3. Créer un nouvel utilisateur
const createUser = (req, res) => {
  const { name, email, role } = req.body;
  if (!estNonVide(name)) {
    return res.status(400).json({ error: "Le nom est obligatoire et ne doit pas être vide" });
  }
  if (!estNonVide(email) || !estEmailValide(email)) {
    return res.status(400).json({ error: "L'email est obligatoire et doit être valide" });
```

```
  }
  const nouvelUtilisateur = { id: prochainId++, name, email, role: role || 'user' };
  users.push(nouvelUtilisateur);
  res.status(201).json({ message: 'Utilisateur créé', user: nouvelUtilisateur });
};
// 4. Supprimer un utilisateur
const deleteUser = (req, res) => {
  const id = Number(req.params.id);
  const userExiste = users.some(u => u.id === id);
  if (!userExiste) {
    return res.status(404).json({ error: `Impossible de supprimer : utilisateur ${id} introuvable` });
  }
  users = users.filter(u => u.id !== id);
  res.status(200).json({ message: `Utilisateur ${id} supprimé avec succès` });
};
module.exports = { getAllUsers, getUserById, createUser, deleteUser };
```

5.2 routers/userRoutes.js

```
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');
// Remarque capitale : Le préfixe '/api/users' sera défini dans server.js.
// Ici, '/' représente donc la racine de la ressource : '/api/users' !
router.get('/', userController.getAllUsers);
router.get('/:id', userController.getUserById);
router.post('/', userController.createUser);
router.delete('/:id', userController.deleteUser);
module.exports = router;
```

5.3 utils/validators.js

```
// utils/validators.js
const estNonVide = (chaine) => typeof chaine === 'string' && chaine.trim().length > 0;
const estEmailValide = (email) => {
  return typeof email === 'string' && email.includes('@') && email.includes('.');
};
module.exports = { estNonVide, estEmailValide };
```

5.4 server.js mis à jour

```
const express = require('express');
const articleRoutes = require('./routers/articleRoutes');
const userRoutes = require('./routers/userRoutes');
const app = express();
const PORT = 3000;
// Middlewares globaux
app.use(express.json()); // indispensable pour lire le corps JSON (req.body)
// Montage des routeurs
app.use('/api/articles', articleRoutes);
app.use('/api/users', userRoutes);
// Route d'accueil pour tester le statut général du serveur
app.get('/', (req, res) => {
  res.json({ message: "API du Blog - Serveur Modulaire Opérationnel (SoC)" });
});
app.listen(PORT, () => {
  console.log(`Serveur modulaire en écoute sur http://localhost:${PORT}`);
```

```
});
```

6. Conclusion

La refactorisation répartit les responsabilités : server.js configure l'application et monte les routeurs, userRoutes.js associe chaque verbe HTTP et chaque URL à une fonction, userController.js contient la logique métier, et validators.js regroupe les contrôles réutilisables. Les tests Postman confirment que l'API users se comporte comme prévu : codes 200, 201, 400 et 404 corrects, filtrage par rôle, et validation des champs name et email. Ajouter une nouvelle ressource revient désormais à créer un contrôleur et un routeur, puis à les monter dans server.js, sans toucher au reste du code.